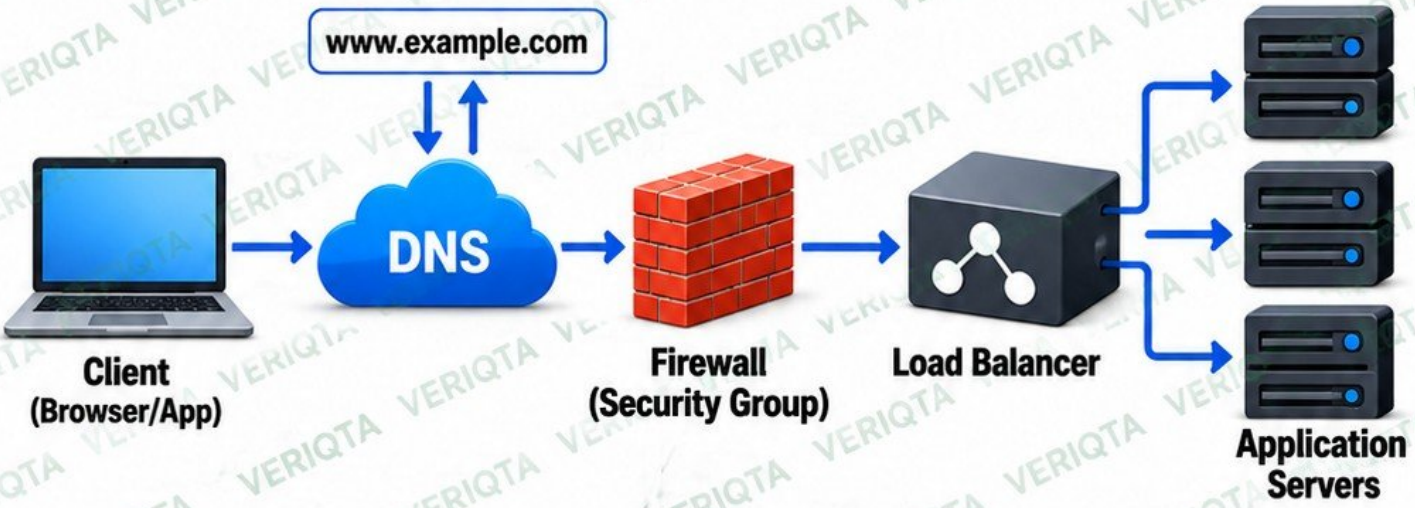
VERIQTA

VERIQTA

The Networking Concepts DevOps Interviews Actually Test





IP Address & Subnetting



TCP vs UDP & Ports



DNS Resolution



HTTP, HTTPS & TLS



Routing & NAT



Firewalls & Security

- Interview-Focused Explanations
- Real-World Diagrams
- Practical Command Examples
- Common Failures and Fixes
- Cloud, Docker & Kubernetes Networking
- Real Interview Questions and Answers
- Production Troubleshooting Scenarios
- Junior Engineer Friendly



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

01 What Happens When You Open a Website?

VERIQTA

What Is This?

When you open a website, your browser follows a series of network steps to find the server, connect securely, send a request, and receive a response. Each step uses different protocols and has a specific job.

High-Level Flow



Browser (Client)	DNS (Domain Name System)	IP (Internet)	TCP / TLS (Secure Connection)	Load Balancer (e.g. ALB/Nginx)	Application Server
1	2	3	4	5	6
You enter a domain name (e.g. example.com) in your browser.	Resolves the domain name to an IP address (e.g. 93.184.216.34).	Your browser connects to the server's IP address using TCP.	TCP establishes a connection. TLS encrypts the connection (HTTPS).	Routes your request to a healthy application server.	Processes your request and sends back a response (e.g. webpage).

Detailed Steps

1. **DNS Lookup**
   Your browser checks local cache, then a DNS resolver to find the IP address of the domain.
2. **TCP Connection**
   The browser starts a TCP handshake with the server's IP and port (80 or 443).
3. **TLS Handshake (if HTTPS)**
   The server presents a certificate. Your browser verifies it and creates an encrypted connection.
4. **HTTP Request**
   Your browser sends an HTTP request (e.g. GET /) over the secure connection.
5. **Load Balancer**
   The request is routed to a healthy backend server based on rules (path, host, etc).
6. **Application Response**
   The application processes the request and sends back an HTTP response (HTML, JSON, etc).

Interview Challenge

A website is unreachable. Which stage would you investigate first, and why?

Expected Answer

Start with DNS resolution.

- If the domain name does not resolve to an IP address, nothing else can work.
- It quickly tells you if the problem is name resolution or something later in the network path.
- Once DNS works, you move to the next stage (TCP, TLS, load balancer, application) based on where the failure occurs.

Common Failures at Each Stage

- DNS: domain not found, wrong IP, expired record
- TCP: connection timeout, connection refused
- TLS: certificate error, handshake failure
- Load Balancer: unhealthy targets, wrong path/port
- Application: 4xx/5xx errors, slow response

Key Takeaway

A website working depends on multiple network components. Interviewers expect you to understand each step, how to verify it, and how to troubleshoot when something fails.

Real World Insight

Use one command at a time to confirm each stage (e.g. dig, curl, openssl) rather than guessing or changing configuration without evidence.

VERIQTA

02 TCP/IP vs OSI: Troubleshooting by Layer

1. OSI Model (7 Layers)

7	Application	HTTP, HTTPS, DNS, SSH, FTP
6	Presentation	TLS, encryption, compression
5	Session	Manages connections
4	Transport	TCP, UDP (ports, reliability)
3	Network	IP (routing, logical addressing)
2	Data Link	Ethernet, MAC address, switches
1	Physical	Cables, radio, electrical signals

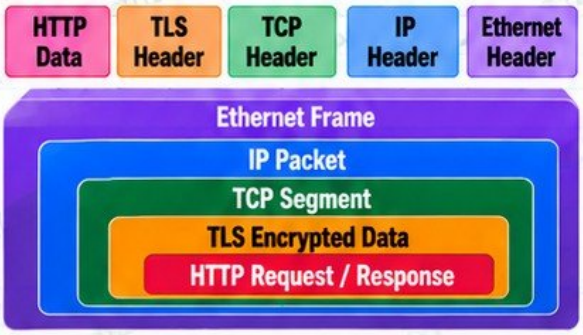
2. TCP/IP Model (4 Layers)

Application	HTTP, HTTPS, DNS, SSH, FTP, TLS
Transport	TCP, UDP
Internet	IP, routing, ICMP
Network Access	Ethernet, Wi-Fi, MAC address, physical media

3. Where Common Protocols Fit

Protocol Technology	OSI Layer	TCP/IP Layer	Purpose
HTTP / HTTPS	7	Application	Web requests and responses
TLS	6	Application	Encrypts data (used by HTTPS)
DNS	7	Application	Resolves domain names to IP
TCP	4	Transport	Reliable, ordered delivery
UDP	4	Transport	Connectionless, faster
IP	3	Internet	Logical addressing and routing
Ethernet	2	Network Access	Local network communication

4. Encapsulation Example
(HTTP over TLS over TCP over IP over Ethernet)



5. Practical Troubleshooting Mapping

Tool / Command	What It Verifies	Layer(s)	Example Command
ping	Basic IP connectivity (uses ICMP)	Network (3)	ping 8.8.8.8
ip route	Routing table and next hop	Network (3)	ip route ip route get 8.8.8.8
ss	Listening ports and active connections	Transport (4)	ss -lntp ss -tuna
curl	Full application request (including TLS/HTTP)	Application (5–7)	curl -v https://example.com

6. Interview Challenge

How do you distinguish a network connectivity failure from an application failure?

Expected Answer (Key Points)

1. Check lower layers first (IP, routing, TCP).
2. If you cannot reach the IP, it is a network issue.
3. If you can reach the IP and port, but the application returns errors (e.g. 4xx/5xx), it is an application issue.
4. Use progressively deeper checks: ping → route → port → TLS → HTTP → application logic.

7. Example Troubleshooting Flow

1. **ping** — Check IP reachability (L3)
2. **ip route** — Check routing and next hop (L3)
3. **ss** — Check if the service is listening (L4)
4. **curl** — Check TLS/HTTP and application (L5–L7)

8. Key Takeaways

- Use the right tool for the right layer.
- Network failures usually prevent you from reaching the IP or port.
- Application failures usually return an HTTP error.
- Understanding layers helps you troubleshoot faster and answer interview questions confidently.

03 IP Addresses, CIDR, and Subnetting

VERIQTA

1. What Are IP Addresses?

- An IP address is a unique number that identifies a device on a network.
- IPv4 uses 32 bits (e.g. 192.168.1.10).
- Devices use IP addresses to send and receive data.

2. IPv4 Address Structure



3. Private vs Public IP Ranges

Type	Range	Used For
Private	10.0.0.0 – 10.255.255.255 172.16.0.0 – 172.31.255.255 192.168.0.0 – 192.168.255.255	Internal networks (e.g. VPC, office, home)
Public	All other IPv4 addresses	Accessible on the internet

4. Subnet Mask and CIDR

- A subnet mask separates the network portion from the host portion.
- CIDR (Classless Inter-Domain Routing) is a shorter notation, e.g. /24 instead of 255.255.255.0.
- A larger / number means more network bits and fewer usable hosts.

CIDR	Subnet Mask	Total Addresses	Usable Hosts
/24	255.255.255.0	256	254
/26	255.255.255.192	64	62
/27	255.255.255.224	32	30

5. Network, Broadcast, and Usable Addresses



For a /26 subnet (64 total addresses):

- Network address: 192.168.10.0
- Usable hosts: 192.168.10.1 – 192.168.10.62
- Broadcast address: 192.168.10.63

6. Interview Challenge

How many usable host addresses are in 192.168.10.0/26?

Answer and Explanation

- /26 leaves 6 bits for hosts (32 – 26 = 6).
- Total addresses = $2^6 = 64$.
- Usable hosts = 64 – 2 = 62.
- Network address: 192.168.10.0
- Usable range: 192.168.10.1 – 192.168.10.62
- Broadcast address: 192.168.10.63

7. Calculating Subnet Ranges (Example)

Find the subnets for 192.168.1.0/24 split into /26:

Subnet	Network Address	Usable IP Range	Broadcast
1	192.168.1.0	192.168.1.1 – 192.168.1.62	192.168.1.63
2	192.168.1.64	192.168.1.65 – 192.168.1.126	192.168.1.127
3	192.168.1.128	192.168.1.129 – 192.168.1.190	192.168.1.191
4	192.168.1.192	192.168.1.193 – 192.168.1.254	192.168.1.255

8. Why AWS Reserves Addresses

In each AWS VPC subnet, the first 4 and last 1 IP addresses are reserved for AWS use.

Address	Purpose
x.x.x.0	Network address (reserved)
x.x.x.1	VPC router (reserved)
x.x.x.2	DNS server (reserved)
x.x.x.3	Future use (reserved)
x.x.x.255	Broadcast address (reserved)

9. Practical Tips

- Use CIDR notation to plan IP ranges.
- Always check network, usable, and broadcast addresses.
- Ensure subnets do not overlap.
- Understand reserved addresses in cloud environments.

10. Common Interview Questions

- What is the difference between a public and private IP address?
- How many usable hosts are in 10.0.0.0/28?
- Which IPs are reserved in an AWS subnet and why?
- How do you calculate subnet ranges quickly?
- Why is subnetting important in cloud architecture?

VERIQTA

04 TCP vs UDP: Reliability and Performance

1. What Are TCP and UDP?

- **TCP** (Transmission Control Protocol)
  Connection-oriented, reliable, ordered delivery.
- **UDP** (User Datagram Protocol)
  Connectionless, faster, no delivery guarantees.
- Both are transport layer protocols (Layer 4) that use port numbers to deliver data to applications.

2. Key Differences

Feature	TCP	UDP
Connection	Connection-oriented	Connectionless
Reliability	Reliable (retransmission)	No guarantee
Ordering	Maintains order	No ordering
Flow control	Yes	No
Speed	Slower (more overhead)	Faster (less overhead)
Use case	Web, SSH, email, DB	DNS, video, gaming, VoIP

3. How TCP Works (Simplified)



- Establishes a connection (3-way handshake).
- Uses sequence numbers and acknowledgements.
- Retransmits lost data.
- Controls data flow so the receiver is not overwhelmed.
- Ensures data arrives in order.

4. How UDP Works (Simplified)



- No connection setup.
- Sends data without waiting for a response.
- No retransmission if data is lost.
- No guarantee of delivery or order.
- Lightweight and fast.
- Used when speed matters more than reliability (e.g. DNS, video streaming, gaming).

5. Where Common Protocols Fit

Protocol	Transport	Default Port	Why
HTTP	TCP	80	Reliable web requests
HTTPS	TCP	443	Reliable + encrypted (TLS)
SSH	TCP	22	Reliable remote access
DNS	UDP (and TCP)	53	Fast queries (UDP), reliability for large responses (TCP)
DHCP	UDP	67/68	Fast configuration
NTP	UDP	123	Small, time-critical data
QUIC (HTTP/3)	UDP	443	Reliable, encrypted, faster (uses UDP)
VoIP (e.g. RTP)	UDP	varies	Real-time, low latency

6. Interview Challenge

Why might DNS use UDP, while SSH uses TCP? Where does QUIC fit?

Expected Answer (Key Points)

1. DNS uses UDP because queries are small and speed matters. It's faster and has lower overhead. If a response is large or zone transfer is needed, DNS can use TCP.
2. SSH uses TCP because it needs a reliable, ordered, secure connection for an interactive session.
3. QUIC (used by HTTP/3) runs over UDP but adds its own reliability, encryption and congestion control, making it faster than traditional TCP + TLS in many cases.

7. Practical: Inspect TCP and UDP Sockets

ss -tuna

- `-t` Show TCP sockets
- `-u` Show UDP sockets
- `-n` Show numeric addresses (no DNS lookup)
- `-a` Show all sockets (listening and established)

Example Output (Simplified)

```
Proto  Local Address      State
tcp    0.0.0.0:22         LISTEN
tcp    0.0.0.0:80         LISTEN
udp    0.0.0.0:53         UNCONN
udp    0.0.0.0:123        UNCONN
```

8. Key Takeaways

- TCP is reliable and ordered, UDP is faster and simpler.
- Use the right protocol for the right use case.
- DNS uses UDP for speed, SSH uses TCP for reliability.
- QUIC uses UDP but provides its own reliability.
- Use ss -tuna to inspect active TCP and UDP sockets.

VERIQTA

05 Ports, Sockets, and Connection Refused

1. What Are Ports and Sockets?

- A **port** is a number that identifies a service on a server (e.g. 80, 443, 8080).
- A **socket** is the combination of an IP address and port (IP:Port) used for communication.
- A **listening socket** waits for incoming connections.
- A **client** uses an **ephemeral port** (temporary port) when making an outgoing connection.

2. How a Connection Works



Client (192.168.1.10) Ephemeral Port 52344)

Server (10.0.0.5 Port 8080 Listening)

3. Common Port Numbers

Port	Protocol	Common Use
22	TCP	SSH (remote access)
53	UDP/TCP	DNS (domain name resolution)
80	TCP	HTTP (web)
443	TCP	HTTPS (web, encrypted)
3306	TCP	MySQL (database)
5432	TCP	PostgreSQL (database)
6379	TCP	Redis (cache)
8080	TCP	Common for web applications

4. Client Ephemeral Ports

- Client operating systems use a temporary port (ephemeral port) for each outgoing connection.
- Linux ephemeral port range (example): 32768 – 60999
- You usually do not choose this port.



Client 192.168.1.10 Port 52344 (ephemeral)

Server 10.0.0.5 Port 8080 (listening)

IP and Port Combination

- The same port can exist on different IPs.
- Example:
  - 127.0.0.1:8080
  - 192.168.1.10:8080
  - 10.0.0.5:8080

5. Socket Binding: 127.0.0.1 vs 0.0.0.0

Bind to 127.0.0.1 (Localhost only)

- Service listens only on the local machine.
- Accessible from the same server using 127.0.0.1 or localhost.
- Not reachable from other devices on the network.



127.0.0.1:8080 (local access only)

Bind to 0.0.0.0 (All network interfaces)

- Service listens on all available network interfaces.
- Accessible from local machine and other devices (if firewall allows it).



0.0.0.0:8080 (local + network access)

6. Practical Commands

Check listening ports

```
ss -lntp
```

- `-l` : show listening sockets
- `-n` : show numeric addresses
- `-t` : TCP sockets
- `-p` : show process name

Test connection using curl

```
curl -v http://localhost:8080
```

- `-v` : verbose output
- Shows connection process and error messages

Check all sockets (TCP and UDP)

```
ss -tuna
```

- `-t` : TCP
- `-u` : UDP
- `-n` : numeric
- `-a` : all sockets

7. Interview Challenge

An application is running, but port 8080 returns "Connection refused". What would you check?

Answer Approach (Steps)

1. Check if the application is actually listening on the port.
2. Verify the IP address it is bound to (127.0.0.1 vs 0.0.0.0).
3. Check if the service is running and healthy.
4. Confirm there is no firewall blocking the port.
5. Test from the correct network (local vs remote).
6. Check application logs for startup errors.

8. Example: Connection Refused Troubleshooting

1	Check if it is listening	`$ ss -lntp | grep 8080` (no output = not listening)
2	Check bind address	`$ ss -lntp` `LISTEN 0 128 127.0.0.1:8080`
3	Test locally	`$ curl -v http://127.0.0.1:8080` (works)
4	Test from another device	`$ curl -v http://<server-ip>:8080` (connection refused)

9. Key Takeaways

- A service can be running but still not reachable on the network.
- Always check what IP address and port the service is listening on.
- "Connection refused" usually means nothing is listening on that IP:port.
- Use ss, curl, and firewall checks to find the real cause.

06 DNS Resolution and Why It Fails

VERIQTA

1. What Is DNS?

- DNS (Domain Name System) translates human readable domain names (e.g. example.com) into IP addresses (e.g. 93.184.216.34).
- It uses a distributed system of resolvers and authoritative nameservers.
- Common record types: A, AAAA, CNAME, MX, TXT, etc.

2. How DNS Resolution Works



3. Common DNS Record Types

Record Type	Purpose	Example
A	Maps a domain to an IPv4 address	example.com → 93.184.216.34
AAAA	Maps a domain to an IPv6 address	example.com → 2001:db8::1
CNAME	Alias to another domain	www → example.com
MX	Mail server for the domain	example.com → mail.example.com
TXT	Text information (SPF, verification)	example.com → "v=spf1 ..."
NS	Authoritative nameserver	example.com → ns1.example.com

4. DNS Components

- Recursive Resolver: receives queries from clients and finds the answer.
- Root Nameserver: points to the correct TLD server.
- TLD Nameserver: points to the domain's authoritative nameserver.
- Authoritative Nameserver: holds the actual records.
- Cache: stores answers for a period of time (TTL).
- TTL (Time To Live): how long a record is cached.
- NXDOMAIN: returned when a domain does not exist.

5. DNS Caching and TTL

- Answers are cached at multiple levels (OS, browser, recursive resolver, CDN, etc).
- TTL controls how long the record stays in cache.
- Low TTL = faster updates, higher DNS queries.
- High TTL = less load, slower propagation.

6. Practical Commands

Use dig

```
dig example.com
dig example.com A
dig example.com AAAA
dig example.com +short
```

Use getent hosts

```
getent hosts
example.com
```

Check resolver config

```
cat /etc/resolv.conf
cat /etc/hosts
```

7. Example: dig Output (Simplified)

```
$ dig example.com
;; ANSWER SECTION:
example.com.      300  IN   A    93.184.216.34
$ dig example.com AAAA
;; ANSWER SECTION:
example.com.      300  IN  AAAA  2606:2800:220:1:248:1893:25c8:1946
```

8. /etc/resolv.conf Example

```
$ cat /etc/resolv.conf
nameserver 10.96.0.10
search default.svc.cluster.local svc.cluster.local
options ndots:5
```

Common in Kubernetes

The Pod uses the cluster DNS service (e.g. 10.96.0.10) and a search domain like svc.cluster.local.

9. Interview Challenge

A domain resolves on your laptop but fails inside a Kubernetes Pod. Why?

10. Expected Answer (Key Points)

1. The Pod is using a different DNS server (check /etc/resolv.conf).
2. The cluster DNS cannot resolve external domains.
3. Missing or incorrect search domain / ndots configuration.
4. NetworkPolicy or firewall blocking DNS (UDP/TCP port 53).
5. The record exists on your local DNS cache but not on the cluster resolver.
6. The domain requires a specific resolver (e.g. corporate DNS).

11. Common DNS Failures and Fixes

Issue	Cause	Fix
NXDOMAIN	Domain does not exist	Check spelling, DNS record
Timeout	DNS server unreachable	Check network, DNS IP, firewall
Wrong IP	Cached old record	Wait for TTL or clear cache
Works on laptop but not in Pod	Different DNS server or network policy	Check /etc/resolv.conf, network policy
Intermittent failures	DNS server overloaded	Use reliable resolver, retry, multiple DNS

12. Key Takeaways

- DNS is essential for every application.
- Understand the components and query flow.
- Use dig, getent and /etc/resolv.conf to troubleshoot.
- Check network policies and firewalls when DNS fails.
- Different environments (laptop, Pod, VM, container) may use different DNS servers.
- Always verify the actual DNS server being used.

07 Routing Tables and Default Gateways

VERIQTA

1. What Is a Routing Table?

- A routing table tells the Linux kernel where to send network traffic.
- Each route has a destination, next hop (gateway), and network interface.
- Linux uses the best matching route based on the longest-prefix matching rule.
- If no specific route matches, it uses the default route (0.0.0.0/0).

2. How Linux Selects a Route (Longest-Prefix Match)

1. Check for exact match (highest prefix)
2. If no match, check less specific routes
3. If no match, use default route (0.0.0.0/0)
4. Send to next hop via the specified interface

Example destination: 8.8.8.8

Route	Prefix Length	Match?
10.0.0.0/24	/24	No
192.168.1.0/24	/24	No
0.0.0.0/0	/0	Yes (use this route)

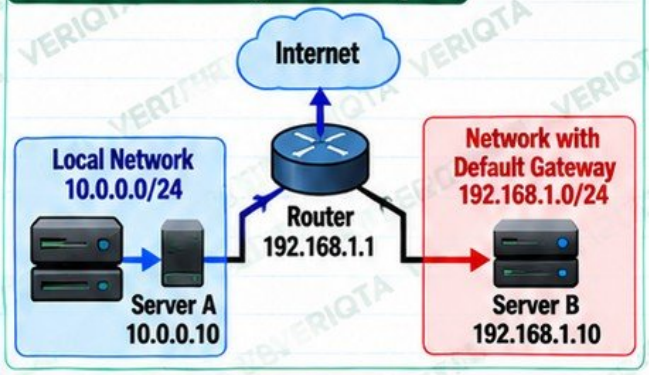
3. Example Routing Table (ip route)

```
$ ip route
default via 192.168.1.1 dev eth0
10.0.0.0/24 dev eth1 proto kernel scope link src 10.0.0.10
192.168.1.0/24 dev eth0 proto kernel scope link src 192.168.1.10
```

Default route (to internet)

Local network route

4. Network Diagram Example



5. Common Routing Table Entries

Destination	Meaning	Next Hop	Interface
10.0.0.0/24	Local network	Direct (on-link)	eth1
192.168.1.0/24	Local network	Direct (on-link)	eth0
0.0.0.0/0	Default route (all other networks)	192.168.1.1	eth0
172.16.0.0/16	Another internal network	10.0.0.1	eth1

6. Practical Commands

Show routing table

```
ip route
```

```
$ ip route
# shows all routes
```

Check route to a specific IP

```
ip route get 8.8.8.8
```

```
$ ip route get 8.8.8.8
# shows which route
# and interface will
# be used
```

Trace the path

```
traceroute 8.8.8.8
```

```
$ traceroute 8.8.8.8
# shows each hop
# to the destination
```

7. Interview Challenge

A server can reach its local subnet but not an external IP. What could be wrong?

Expected Answer (Key Points)

1. Missing or incorrect default route (0. 0.0.0/0).
2. Incorrect or unreachable default gateway.
3. DNS may work, but there is no internet route.
4. Firewall or network ACL blocking outbound traffic.
5. NAT not configured (in cloud environments).
6. Asymmetric routing or return traffic being blocked.

8. Troubleshooting Steps

1. Check the routing table: ip route
2. Verify the default gateway is reachable: ping 192.168.1.1
3. Test external IP reachability: ping 8.8.8.8
4. Check the route used: ip route get 8.8.8.8
5. Trace the path: traceroute 8.8.8.8
6. Check firewall rules and NAT (if in cloud).

9. Example: ip route get Output

```
$ ip route get 8.8.8.8
8.8.8.8 via 192.168.1.1 dev eth0 src 192.168.1.10
    cache
```

- 8.8.8.8 → Destination IP
- via 192.168.1.1 → Next hop (default gateway)
- dev eth0 → Network interface used
- src 192.168.1.10 → Source IP address for this route

VERIQTA

08 NAT, SNAT, DNAT, and Internet Access

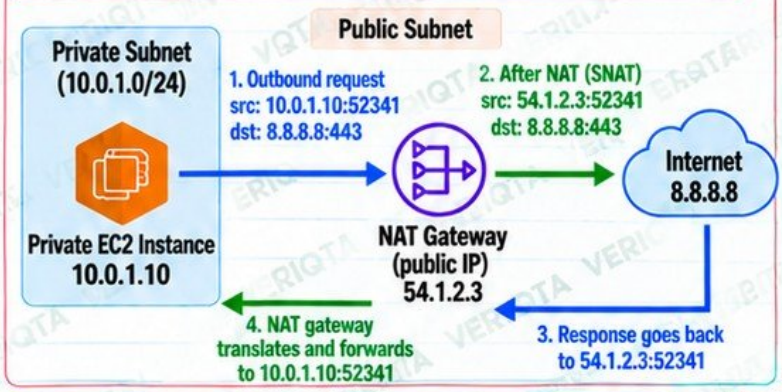
1. What Is NAT?

- NAT (Network Address Translation) rewrites IP addresses and/or port numbers as traffic passes through a device (e.g. router, NAT gateway).
- It allows private IP addresses to communicate with external networks.
- It keeps track of connections using a connection tracking table.
- Common types: SNAT, DNAT, and PAT (NAT overload).

2. Types of NAT

Type	What It Does	Use Case	Example
SNAT	Changes source IP (and often port)	Outbound internet access	Private IP → Public IP
DNAT	Changes destination IP (and often port)	Inbound access to internal service	Public IP → Private IP
PAT (NAT Overload)	Many private IPs share one public IP using different ports	Multiple instances to the internet	192.168.1.10:52341 → 54.1.2.3:443

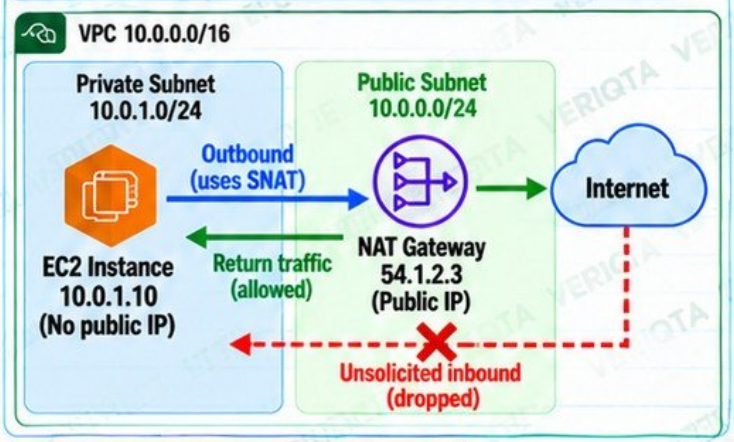
3. How Outbound Internet Access Works (SNAT)



4. Why Inbound Connections Do Not Work

- A NAT gateway allows outbound connections because it creates a connection tracking entry when your instance initiates the request.
- It only allows return traffic for established connections.
- It does not have a mapping for unsolicited inbound traffic, so it drops it.
- To allow inbound access to a private instance, you need a public load balancer, bastion host, or port forwarding device (DNAT) with a static mapping.

5. NAT in AWS VPC (Example Architecture)



6. Interview Challenge

Why can a private EC2 instance access the internet through NAT but not accept unsolicited inbound connections through that NAT?

Expected Answer (Key Points)

1. The NAT gateway performs SNAT for outbound traffic.
2. It creates a connection tracking entry for each outbound flow.
3. Return traffic is allowed because it matches an existing entry.
4. Unsolicited inbound traffic has no existing mapping, so it is dropped.
5. To accept inbound connections, you need a public load balancer, bastion host, or DNAT with a static port mapping.

7. Practical Commands (Linux)

Check routing table

```
ip route
```

```
default via 10.0.0.1 dev eth0
10.0.0.0/24 dev eth0 proto kernel
192.168.1.0/24 dev eth1 proto kernel
```

See route to a specific IP

```
ip route get 8.8.8.8
```

```
8.8.8.8 via 10.0.0.1 dev eth0
src 10.0.0.1.10
```

Test external connectivity

```
traceroute 8.8.8.8
```

```
1  10.0.0.1   1.2 ms
2  54.1.2.3   5.6 ms
3  8.8.8.8    12.4 ms
```

8. Key Takeaways

- NAT changes IP addresses and/or ports.
- SNAT is used for outbound internet access.
- DNAT is used for inbound access to internal services.
- Return traffic works because of connection tracking.
- Unsolicited inbound connections are dropped.
- Use a load balancer or bastion host for inbound access.

09 Firewalls, Security Groups, and NACLs

VERIQTA

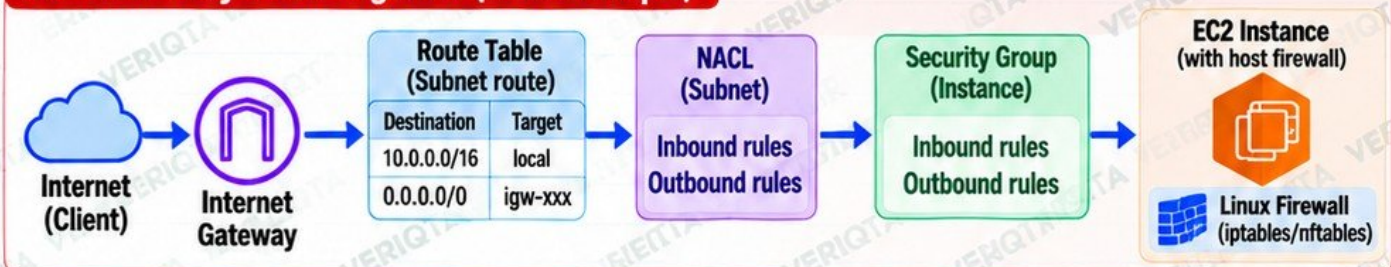
1. What Are Firewalls, Security Groups, and NACLs?

- A **firewall** filters network traffic based on rules (e.g. allow or deny).
- **Security groups (SG)** are stateful instance-level firewalls in AWS.
- **Network ACLs (NACL)** are **stateless** subnet-level firewalls in AWS.
- They control inbound and outbound traffic using IP addresses, ports, and protocols.

2. Stateful vs Stateless Filtering

Feature	Stateful (Security Group)	Stateless (NACL)
Tracks connections	Yes	No
Return traffic	Automatically allowed	Must be explicitly allowed
Rule type	Allow only	Allow and Deny
Applied to	Instance (ENI)	Subnet
Use case	Instance-level control	Subnet-level control
Default behavior	Deny all (until allowed)	Allow all (unless denied)

3. How They Work Together (AWS Example)



4. Inbound and Outbound Rules Example

Control	Direction	Example Rule	Purpose
Security Group (stateful)	Inbound	Allow TCP 443 from 0.0.0.0/0	Allow HTTPS to instance
Security Group (stateful)	Outbound	All traffic (default)	Allow return traffic automatically
NACL (stateless)	Inbound	Allow TCP 443 from 0.0.0.0/0	Allow HTTPS to subnet
NACL (stateless)	Outbound	Allow TCP 1024–65535 to 0.0.0.0/0	Allow return traffic (ephemeral ports)

5. Linux Firewall Example (iptables)

Allow incoming HTTPS (port 443)

```
sudo iptables -A INPUT -p tcp --dport 443 -j ACCEPT
```

Check firewall rules

```
sudo iptables -L -n -v
```

6. Key Points

- Security groups are stateful: return traffic is allowed automatically.
- NACLs are stateless: you must allow both inbound and outbound.
- A host firewall (iptables/nftables) can still block traffic.
- All layers (route table, NACL, security group, host firewall, application) must allow the traffic.

7. Interview Challenge

A security group allows port 443, but clients still time out. What other controls would you inspect?

8. Troubleshooting Steps

1. Check the route table (is there a route to the internet or target?).
2. Check the NACL (are inbound and outbound rules correct?).
3. Check the security group (source IP, port, protocol).
4. Check the host firewall (iptables/nftables).
5. Check the application listener (is the service running?).
6. Check the logs (VPC Flow Logs, instance logs, application logs).

9. Example: Traffic Flow for HTTPS



10. Expected Answer (Key Points)

1. Check the NACL (it might be blocking return traffic).
2. Check the route table (missing route to internet or target).
3. Check the host firewall (iptables/nftables).
4. Check the application (is it actually listening on port 443?).
5. Check external factors (load balancer, WAF, network ACLs, ISP).
6. Use VPC Flow Logs and instance logs to identify where the traffic is dropped.

10 HTTP, HTTPS, TLS, and Status Codes

1. What Are HTTP and HTTPS?

- **HTTP** (Hypertext Transfer Protocol) is the protocol used to send requests and receive responses between a client and a server.
- **HTTPS** is HTTP over TLS (Transport Layer Security), which encrypts the traffic and verifies the server's identity using a certificate.
- HTTPS protects data in transit and prevents eavesdropping and tampering.

2. How a Secure Request Works (HTTPS)



3. Example HTTP Request and Response

HTTP Request (from client)

```
GET /index.html HTTP/1.1
Host: example.com
User-Agent: curl/8.0.1
Accept: text/html
```

HTTP Response (from server)

```
HTTP/1.1 200 OK
Date: Wed, 18 Oct 2026 12:00:00 GMT
Content-Type: text/html
Content-Length: 1256

<html>...</html>
```

4. Common HTTP Status Codes

Code	Meaning	Example Use
200	OK	Request succeeded
301	Moved Permanently	Redirect to new URL
400	Bad Request	Invalid request
401	Unaathorized	Authentication required
403	Forbidden	Access denied
404	Not Found	Resource not found
429	Too Many Requests	Rate limited
500	Internal Server Error	Server error
502	Bad Gateway	Invalid response from upstream server
503	Service Unavailable	Server temporarily down
504	Gateway Timeout	Upstream server timeout

5. TLS Components

- **TLS Handshake:** negotiates cipher suite and encrypts the connection.
- **Certificate:** proves the server's identity (issued by a CA).
- **SNI (Server Name Indication):** sends the domain name during the handshake (important for hosting multiple websites).
- **Certificate Validation:** client checks the certificate chain, expiry date, and domain name.

6. TLS Handshake Process



7. Practical Comnands

Check HTTPS headers

```
curl -vI https://example.com
```

What to look for:
- TLS handshake details
- Certificate information
- HTTP status code
- Response headers

Inspect TLS certificate

```
openssl s_client -connect
example.com:443 -servername
example.com
```

What to look for:
- Certificate subject and issuer
- Validity dates
- SNI (servername)
- Handshake success or errors

8. Interview Challenge

What is the difference between a TLS handshake failure, HTTP 502, and HTTP 504?

9. Example Comparison

Issue	What It Means	Common Causes
TLS handshake failure	Connection fails during TLS handshake (no HTTP response)	Invalid certificate, wrong domain (SNI), expired certificate, unsupported TLS version or cipher
HTTP 502 Bad Gateway	Gateway received an invalid response from upstream server	Upstream server down, misconfigured load balancer, application error
HTTP 504 Gateway Timeout	Gateway did not receive a timely response from upstream server	Upstream server slow or unresponsive, network issues, timeout configuration

10. Key Takeaways

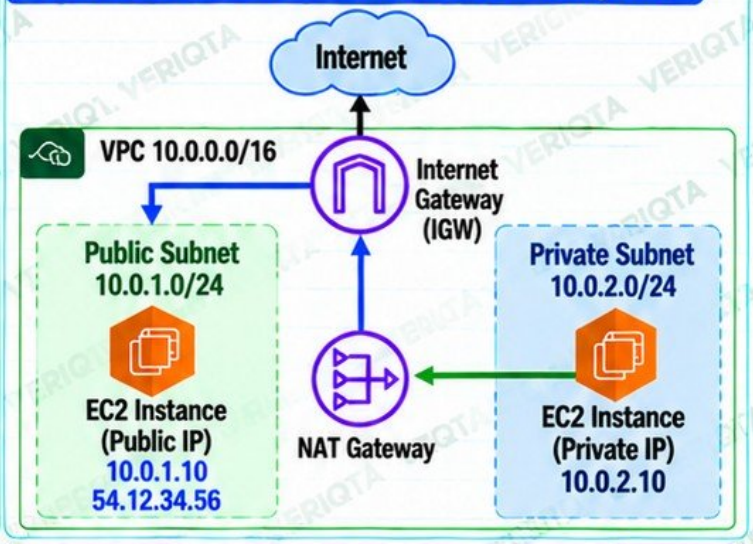
- HTTP sends data in plain text; HTTPS encrypts it with TLS.
- TLS verifies the server using a certificate and uses SNI for the domain name.
- Status codes help you quickly identify where a problem is occurring.
- A TLS handshake failure happens before any HTTP request.
- HTTP 502 means an invalid response from the upstream server.
- HTTP 504 means a timeout waiting for the upstream server.
- Use curl and openssl to inspect HTTPS behavior and debug issues.

11 AWS VPC Networking Interview Scenarios

1. Key VPC Networking Components

- **VPC:** isolated virtual network in AWS with a CIDR block.
- **Subnets:** divide the VPC into smaller ranges (public or private).
- **Internet Gateway (IGW):** connects public subnets to the internet.
- **NAT Gateway:** allows private subnet instances to access the internet.
- **Route Tables:** control where traffic is sent.
- **Security Groups:** stateful instance-level firewall.
- **Network ACLs:** stateless subnet-level firewall.

2. AWS VPC Network Architecture (Diagram)



3. Example Route Tables

Public Subnet Route Table

Destination	Target
10.0.0.0/16	local
0.0.0.0/0	igw-xxxx

Private Subnet Route Table

Destination	Target
10.0.0.0/16	local
0.0.0.0/0	nat-xxxx

4. Public vs Private Subnet Comparison

Feature	Public Subnet	Private Subnet
Internet access	Yes (via IGW)	No (via NAT Gateway)
Route table default route	0.0.0.0/0 → IGW	0.0.0.0/0 → NAT
Public IP	Can have public IP	No public IP (uses private IP)
Typical use	Bastion, ALB, web servers	Application servers, databases
Direct inbound from internet	Yes (if SG/NACL allow)	No (use ALB, bastion, or VPN)

5. Security Boundaries

Security Group (SG)	• Stateful (return traffic allowed automatically). • Applied to ENI / instance / load balancer. • Allows or denies traffic based on IP, port, protocol.
Network ACL (NACL)	• Stateless (must allow both inbound and outbound). • Applied at subnet level. • Supports allow and deny rules with rule numbers.

6. Common VPC Interview Scenarios

Scenario	What to Check
EC2 has a public IP but no internet	Route table, IGW attached, SG outbound, NACL, host firewall, DNS.
Private EC2 cannot reach internet	NAT gateway, private route table, SG outbound, NACL, DNS.
Cannot access RDS from EC2	Security group (both sides), NACL, correct subnet/route, endpoint/VPN.
ALB not reachable from internet	ALB SG, subnet route table, IGW, NACL.
App in private subnet needs external API	NAT gateway or VPC endpoint, route table, SG, NACL.

7. Interview Challenge

An EC2 instance has a public IP but cannot access the internet. What must you verify?

8. Troubleshooting Steps

1. Check if the instance can reach the IGW: `ping 8.8.8.8`
2. Verify the public subnet route table (0.0.0.0/0 → IGW).
3. Check the security group outbound rules.
4. Check the network ACL (allow outbound and inbound).
5. Verify DNS resolution (`nslookup` or `dig`).
6. Check the instance OS firewall (iptables/nftables).
7. Check VPC flow logs for dropped traffic.

9. Useful Commands (EC2 Linux)

Check instance public IP
```
curl -s http://checkip.amazonaws.com
```

Test internet connectivity
```
ping 8.8.8.8
```

Test DNS resolution
```
nslookup example.com
```

Check route table in instance
```
ip route
```

Check outbound connection (HTTPS)
```
curl -vI https://example.com
```

10. Key Takeaways

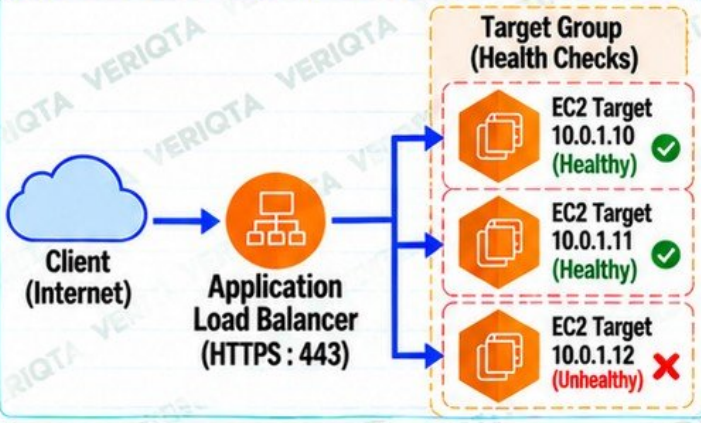
- ✓ VPC CIDR defines the IP address range.
- ✓ Public subnets use an IGW for internet access.
- ✓ Private subnets use a NAT gateway for outbound internet access.
- ✓ Route tables determine the path for traffic.
- ✓ Security groups are stateful; NACLs are stateless.
- ✓ Always verify all layers: route table, IGW/NAT, SG, NACL, host firewall, DNS, and application.

12 Load Balancers, Reverse Proxies, and Health Checks

1. What Are Load Balancers and Reverse Proxies?

- **Load balancer** distributes traffic across multiple backend servers (targets).
- **Layer 4 (L4)** works at the transport layer (TCP/UDP) and forwards traffic based on IP and port.
- **Layer 7 (L7)** works at the application layer (HTTP/HTTPS) and can route based on host, path, headers, etc.
- **Reverse proxy** sits in front of backend servers, accepts client requests, and forwards them to targets (e.g. Nginx, Apache, AWS ALB).
- **Health checks** verify that targets are healthy before sending traffic to them.

2. Architecture Example (AWS Application Load Balancer)



3. Layer 4 vs Layer 7 Load Balancing

Feature	Layer 4 (L4)	Layer 7 (L7)
Protocol	TCP/UDP	HTTP/HTTPS
Routing based on	IP and port	Host, path, headers, cookies, query strings
Example (AWS)	Network Load Balancer (NLB)	Application Load Balancer (ALB)
Use case	High performance, low latency	Advanced routing, web applications

4. Health Check Process

1. Load balancer sends health check request (HTTP/HTTPS/TCP)
2. Target responds with expected status code (e.g. 200 OK)
3. Target marked Healthy or Unhealthy based on response

Common Health Check Settings (ALB)

Setting	Description
Protocol	HTTP or HTTPS
Path	URL path to check (e.g. /health)
Port	Target port (or traffic port)
Success codes	HTTP status codes (e.g. 200–399)
Interval	Time between checks (e.g. 30 seconds)
Timeout	How long to wait for a response (e.g. 5 seconds)
Unhealthy threshold	Number of failed checks before marked unhealthy

5. Reverse Proxy Example (Nginx)



6. Common Causes of Unhealthy Targets

- Health check path returns 404 or 500.
- Health check port is different from the application port.
- Security group or NACL blocking the load balancer.
- Application only responds to certain host headers.
- Application requires authentication.
- High response time (timeout).
- Application is not running or crashing.

7. Interview Challenge

A target responds successfully to manual requests but remains unhealthy behind an AWS Application Load Balancer. Why?

8. Troubleshooting Steps

1. Verify health check path, port, and protocol settings.
2. Check target response code (should match success codes).
3. Check target reachability from the load balancer (security group, NACL).
4. Test the health check path manually (curl -v http://<target-ip>:<port>/<path>).
5. Check application logs for errors.
6. Confirm host header requirements (if using virtual hosts).

9. Practical Commands

Test target directly

```
curl -v http://10.0.1.10/health
```

Test with host header (if required)

```
curl -v -H "Host: example.com" \
http://10.0.1.10/health
```

Check open port on target

```
nc -zv 10.0.1.10 80
```

10. Key Takeaways

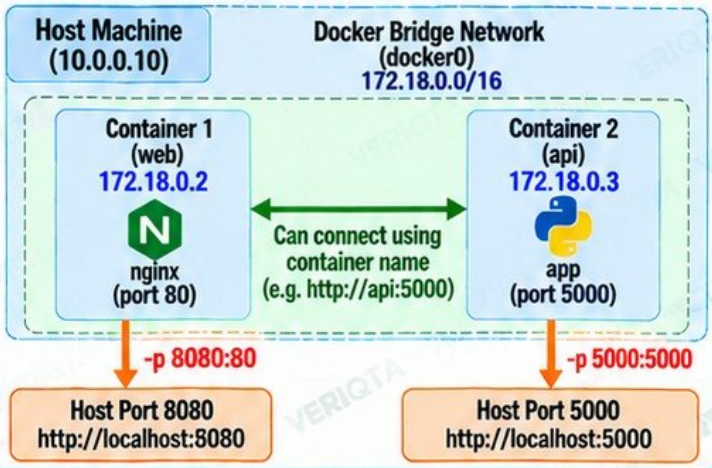
- Load balancers distribute traffic and rely on health checks.
- Layer 4 uses IP/port, Layer 7 uses application data.
- Health checks must match the application's behavior.
- Targets can be reachable but still unhealthy.
- Always verify security groups, NACLs, and application logs.

13 Docker Networking: Why Containers Cannot Connect

1. Key Docker Networking Concepts

- **Bridge network:** default network for standalone containers on the same host.
- **Container DNS:** provides automatic name resolution between containers on user-defined networks.
- **Published ports:** map container ports to the host (for external access).
- **Network namespaces:** each container has its own network stack and isolated localhost.
- **Host vs container localhost:** localhost inside a container refers only to that container, not the host and not other containers.

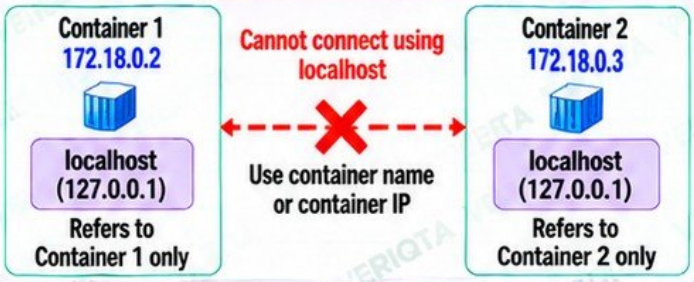
2. Docker Networking Architecture (Bridge Network)



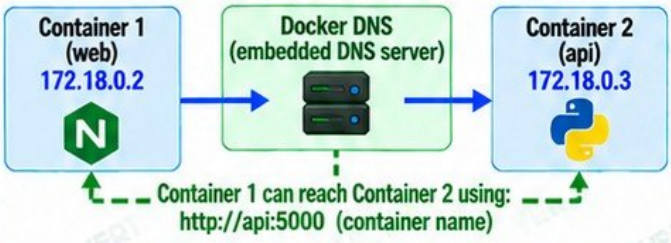
3. Default vs User-Defined Bridge Networks

Feature	Default Bridge (bridge)	User-Defined Bridge (custom)
Network name	bridge	e.g. myapp-net
Automatic DNS (name resolution)	No	Yes (use container name)
Recommended	Not recommended for multi-container apps	Recommended
Communication	Use IP address	Use container name (e.g. http://api:5000)

4. Why Containers Cannot Connect Using localhost



5. Container Name Resolution (User-Defined Network)



6. Practical Commands

1	List Docker networks	`docker network ls`
2	Inspect a network	`docker network inspect myapp-net`
3	Execute a command inside a container	`docker exec -it web bash`
4	Test connectivity between containers	`docker exec -it web curl http://api:5000`
5	Check container DNS resolution	`docker exec -it web nslookup api`

7. Interview Challenge

Two containers are running, but one cannot connect to the other using localhost. Explain why.

8. Troubleshooting Steps

1. Check that both containers are on the same user-defined network.
2. Verify container name resolution (nslookup api).
3. Check the target service is running and listening on the correct port.
4. Test connectivity using the container name and container IP.
5. Verify published ports if accessing from the host.
6. Check firewall rules and network policies (if using Docker in a secured environment).

9. Common Issues and Fixes

Issue	Why It Happens	Fix
Cannot connect using localhost	localhost is container's own network namespace	Use container name or container IP
Container name not resolved	Using default bridge network	Use a user-defined network
Connection refused	Application not running or wrong port	Check service and port inside the target container
Timeout	Different networks or firewall rules	Ensure same network and verify firewall
Cannot access from host	Port not published	Use -p hostPort:containerPort

10. Key Takeaways

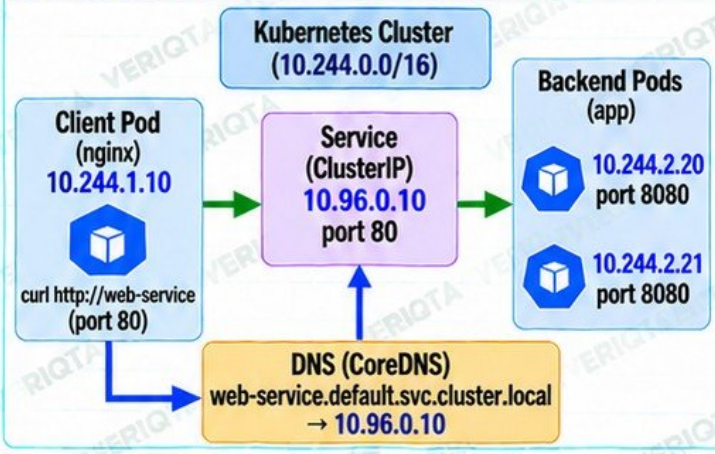
- Each container has its own network namespace and its own localhost.
- Use user-defined bridge networks for automatic DNS and easy container-to-container communication.
- Use container names instead of IP addresses.
- Publish ports only when you need access from the host or external clients.
- Always verify network settings, service status, and firewall rules when containers cannot connect.

14 Kubernetes Networking: Pods, Services, and DNS

1. Key Kubernetes Networking Concepts

- **Pod IP:** each pod gets a unique IP address in the cluster.
- **Services:** provide stable networking and load balancing to a set of pods.
- **ClusterIP:** internal virtual IP accessible within the cluster.
- **Service discovery:** pods can reach services using DNS names.
- **Endpoints:** list of pod IPs behind a service.
- **CNI (Container Network Interface):** handles pod networking, routing, and network policies.
- **Network path:** client pod → service (ClusterIP) → endpoints (pod IPs).

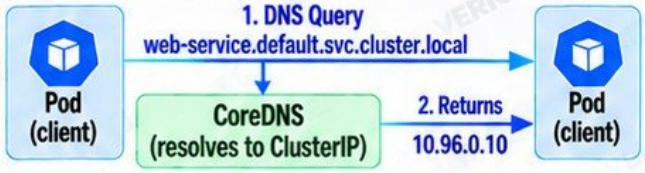
2. Kubernetes Networking Architecture (Service Flow)



3. Kubernetes Service Types

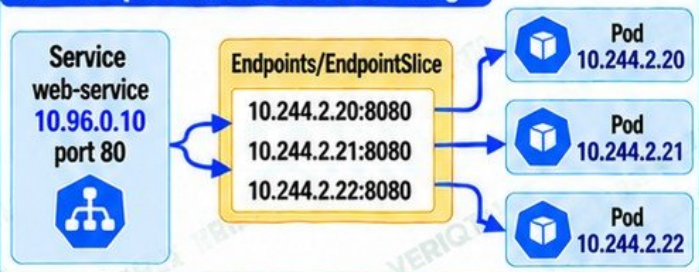
Service Type	IP Address	Accessible From	Use Case
ClusterIP	Internal (e.g. 10.96.0.10)	Inside cluster only	Default, internal communication
NodePort	Node IP:Port (e.g. 30080)	Inside and outside cluster (via node IP)	Testing, simple exposure
LoadBalancer	External IP (via cloud provider)	Outside cluster	Production exposure
ExternalName	DNS CNAME	Inside cluster (resolves to external)	Access external services

4. DNS and Service Discovery



Pod can now connect to the service using:
http://web-service.default.svc.cluster.local:80
(or http://10.96.0.10:80)

5. Endpoints and Traffic Routing



6. Practical Commands

1. Get services in the namespace
```
kubectl get svc
```
2. Get endpointslices for a service
```
kubectl get endpointslices
```
3. Describe the service (details)
```
kubectl describe svc web-service
```
4. Check DNS resolution from a pod
```
kubectl exec -it nginx -- nslookup web-service
```
5. Test connectivity to the service
```
kubectl exec -it nginx -- curl -v http://web-service:80
```

7. Interview Challenge

A Kubernetes Service exists, but traffic never reaches the application. What would you inspect?

8. Troubleshooting Steps

1. Check if the service exists and has the correct type and port (kubectl get svc, kubectl describe svc).
2. Check endpoints/endpointslices (are there ready pod IPs?).
3. Verify pod health and readiness (kubectl get pods, logs).
4. Check the pod selectors match the service (labels).
5. Test DNS resolution from a pod (nslookup or dig).
6. Test direct connection to pod IP and service ClusterIP.
7. Check network policies, CNI, and firewall rules (if blocking traffic).

9. Common Issues and Causes

Issue	Possible Cause
Service has no endpoints	Selector does not match any pods or pods not ready
DNS resolution fails	CoreDNS issue or wrong service name/namespace
Connection refused	Application not running or wrong port/targetPort
Timeout	NetworkPolicy blocking traffic
Works with pod IP but not service	Service selector/endpoint issue or kube-proxy/CNI issue
Intermittent failures	Unhealthy pods, readiness probe failing
Cannot resolve service across namespace	Use full DNS name (service.namespace.svc.cluster.local)

10. Key Takeaways

- Pods have unique IPs and communicate over the cluster network (CNI).
- Services provide stable IP and DNS name for accessing pods.
- Use endpoints/endpointslices to verify which pods are behind a service.
- Service discovery uses DNS (CoreDNS).
- Always check selectors, pod readiness, network policies, and DNS when traffic fails.

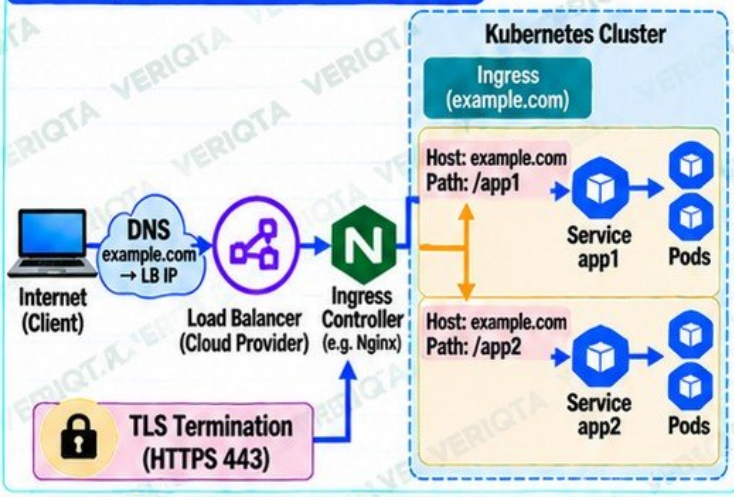
15 Kubernetes Ingress and NetworkPolicy

VERIQTA

1. Key Concepts

- **Ingress:** exposes HTTP/HTTPS services to external traffic using host and path rules.
- **Ingress Controller:** watches Ingress resources and configures a load balancer/reverse proxy (e.g. Nginx).
- **Host/Path Routing:** routes traffic based on domain name and URL path.
- **TLS Termination:** decrypts HTTPS at the ingress controller (or pass-through to the service).
- **NetworkPolicy:** controls which pods can communicate (ingress and egress) at the network level.
- **Namespace Isolation:** use NetworkPolicy to isolate applications between namespaces.
- **CNI Enforcement:** network policies are enforced by the CNI plugin (e.g. Calico, Cilium, Weave).

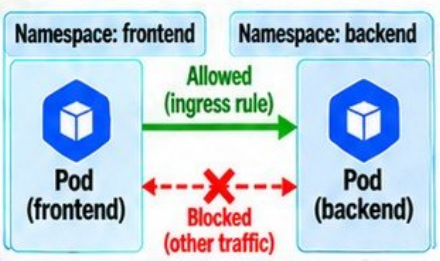
2. Ingress Architecture (Flow)



3. Ingress Resource Example (Host and Path Routing)

Rule	Host	Path	Backend Service	Use Case
1	example.com	/	web-service	Main site
2	example.com	/api	api-service	API traffic
3	app.example.com	/	app-service	Separate app

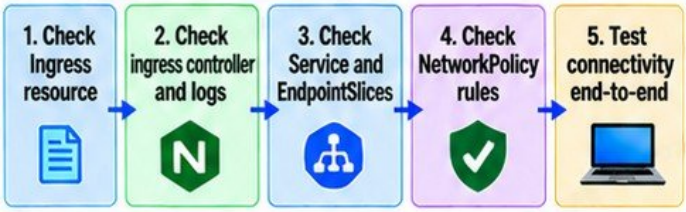
4. NetworkPolicy Example (Namespace Isolation)



NetworkPolicy Rules

- ✓ Allow frontend → backend on port 8080
- ✗ Block all other traffic
- ✓ Enforce isolation between namespaces (using CNI plugin)

5. Troubleshooting Flow



6. Practical Commands

1	Get ingress resources	`kubectl get ingress -A`
2	Describe ingress	`kubectl describe ingress <name>`
3	Check ingress controller pods	`kubectl get pods -n ingress-nginx`
4	Check ingress controller logs	`kubectl logs -n ingress-nginx <pod>`
5	Get service and endpoints	`kubectl get svc, endpointslices`
6	Describe service	`kubectl describe svc <name>`
7	Check network policies	`kubectl get networkpolicy -A`
8	Describe network policy	`kubectl describe networkpolicy <name>`

7. Interview Challenge

An Ingress route exists, but external traffic cannot reach the application. Where do you start?

8. Common Issues and Causes

Issue	Possible Cause	What to Check
Cannot reach Ingress URL	DNS, load balancer, or ingress controller issue	DNS record, LB health, ingress controller pods and logs
404 Not Found	Incorrect host/path rule or service name	Ingress rules, service name, path, and backend
502/503 Error	No ready endpoints or unhealthy pods	EndpointSlices, pod readiness, service selector
Connection timed out	NetworkPolicy blocking traffic	NetworkPolicy rules (ingress/egress)
TLS/SSL errors	Certificate or TLS config issue	Ingress TLS configuration and secret
Works inside cluster but not externally	Load balancer / security group / firewall	Cloud LB, SG/NACL, ingress controller service type

9. Key Takeaways

- ✓ Ingress provides external access using host and path rules.
- ✓ Ingress controller (e.g. Nginx) configures load balancer and routes traffic to services.
- ✓ TLS can be terminated at the ingress controller.
- ✓ Services and EndpointSlices must have healthy pod endpoints.
- ✓ NetworkPolicy controls pod-to-pod traffic and can block ingress or egress.
- ✓ Use namespace isolation with NetworkPolicy for security.
- ✓ Always verify ingress, controller logs, service endpoints, and network policies when traffic fails.
- ✓ Test the full path: client → DNS → load balancer → ingress → service → pod.

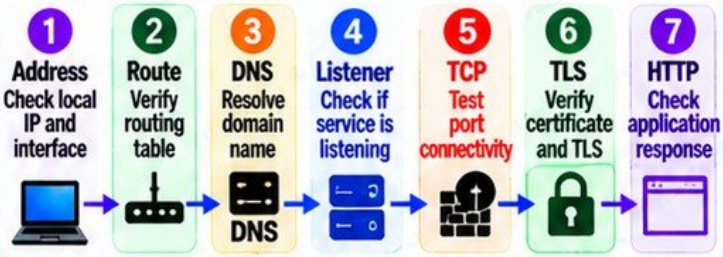
16 Linux Network Troubleshooting Toolkit

VERIQTA

1. Key Network Troubleshooting Tools

Tool	What It Does	What It Proves	What It Cannot Prove
ip	View IP addresses, routes, links	Local network config and routing	Does not prove remote reachability
ss	Check listening ports and connections	What is listening and current connections	Does not test external network path
dig	Query DNS records	DNS resolution and returned IP	Does not prove service is reachable
curl	Test HTTP/HTTPS	Application responds (headers, status)	Does not show network path details
ping	Test ICMP reachability	Basic network connectivity	May be blocked, does not prove port or service
traceroute	Show network path (hops)	Where traffic is going	May be blocked or show limited hops
tcpdump	Capture and analyze packets	Actual network traffic and packets	Does not automatically explain the problem

2. Network Troubleshooting Flow



3. Common Command Examples

1	Check IP address and interfaces	`ip addr show`
2	Check routing table	`ip route show`
3	Check DNS resolution	`dig example.com`
4	Check if a port is listening (local)	`ss -tulnp | grep :80`
5	Test connectivity to a host	`ping -c 4 example.com`
6	Trace network path	`traceroute example.com`
7	Test HTTP response	`curl -vI http://example.com`
8	Test HTTPS and TLS details	`curl -vI https://example.com`
9	Capture packets on an interface	`sudo tcpdump -i eth0 host example.com`

4. Example: Checking a Web Service (End-to-End)

1	Check local IP	`ip addr show`
2	Check route to destination	`ip route get 93.184.216.34`
3	Resolve domain name	`dig example.com`
4	Check if port is open	`nc -zv example.com 80`
5	Check TLS (HTTPS)	`openssl s_client -connect example.com:443`
6	Check HTTP response	`curl -vI https://example.com`

5. Using tcpdump to Investigate Traffic

```
sudo tcpdump -i eth0 host 93.184.216.34
```

- Captures packets to and from a specific host.
- Helps confirm if traffic is leaving and if a response is returning.
- Useful for investigating dropped traffic, timeouts, and unexpected behavior.

Read the Output

- Look for SYN packets (outgoing).
- Look for SYN-ACK packets (incoming).
- No response = possible network, firewall, or routing issue.

6. Interview Challenge

A server is unreachable. Demonstrate a safe diagnostic sequence without randomly changing firewall rules.

7. Example Output (What to Look For)

```
$ ping -c 4 example.com
64 bytes from 93.184.216.34: icmp_seq=1 ttl=57 time=12.3 ms
```

```
$ traceroute example.com
1 10.0.0.1 2.1 ms   2 172.16.1.1 5.4 ms   3 93.184.216.34  12.8 ms
```

```
$ curl -vI https://example.com
HTTP/2 200
server: nginx
content-type: text/html
```

8. Common Issues and What They Mean

Symptom	Possible Cause	Tool to Use
Cannot resolve domain name	DNS issue or wrong DNS server	dig
Ping fails	Network, routing, or firewall blocking ICMP	ping, traceroute
Connection timed out	Firewall, security group, or service not listening	ss, nc, tcpdump
Connection refused	Service not running	ss, nc
TLS handshake fails	Certificate issue, TLS version/cipher	openssl, curl
HTTP 4xx/5xx error	Application issue	curl
No traffic seen in tcpdump	Routing or firewall blocking traffic	tcpdump, ip route

9. Key Takeaways

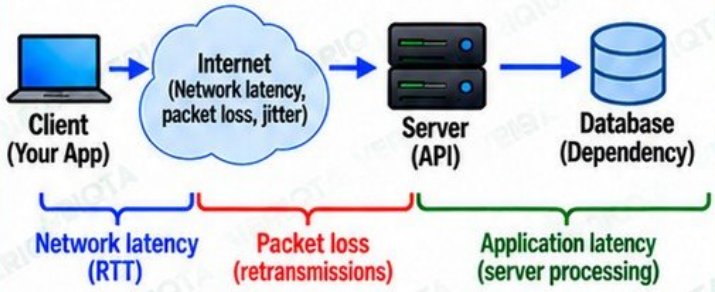
- Follow a structured diagnostic sequence: Address → Route → DNS → Listener → TCP → TLS → HTTP.
- Understand what each tool proves and what it cannot prove.
- Use non-intrusive checks before making any changes.
- Verify both client-side and server-side evidence.
- Check DNS, routing, firewall rules, and service status.
- Use tcpdump to confirm actual network traffic.
- Avoid randomly changing firewall rules.
- Document findings at each step.

17 Timeouts, Packet Loss, and Network Latency

1. Key Concepts

- **Round-trip time (RTT):** time for a packet to go to the destination and back.
- **Packet loss:** percentage of packets that do not reach the destination.
- **Jitter:** variation in latency over time.
- **TCP retransmissions:** packets resent due to loss or no acknowledgement.
- **Connection timeouts:** occur when a response is not received within the expected time.
- **Application latency:** total time to process a request (network + server processing).

2. How Network Delays Affect an Application (Flow)

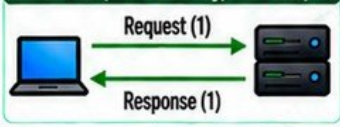


3. Common Network Metrics

Metric	What It Means	Typical Good Range	What High Values Indicate
Latency (RTT)	Time for a packet to travel and return	< 50 ms (local) < 100 ms (global)	Network congestion, distance, or routing issues
Packet loss	% of packets not reaching destination	0% (ideal) < 1% (acceptable)	Network congestion, unstable links, or device issues
Jitter	Variation in latency	Low and stable	Network instability, congestion, or changing routes
Retransmissions	Packets sent again due to loss/no ACK	Low	Packet loss, poor network quality, or overloaded endpoints
Timeout	No response within expected time	Should be minimal	Network issues, firewall blocking, or slow application

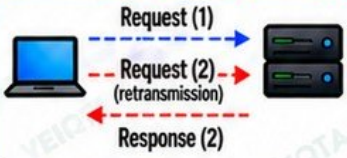
4. Normal vs Problematic Network Behavior

Normal (Low Latency, No Loss)



- Low RTT (e.g. 20 ms)
- 0% packet loss
- No retransmissions
- Fast application response

Problem (High Latency / Loss)



- High RTT (e.g. 500 ms)
- Packet loss (e.g. 5%)
- Retransmissions
- Slow or timed out application response

5. Practical Commands and Examples

Test latency and packet loss (but do not rely only on ICMP)

```
ping -c 5 example.com
traceroute example.com
mtr -rw example.com
```

Check TCP connection and retransmissions

```
ss -ti dst example.com:443
ss -s
sudo tcpdump -ni eth0 host example.com
```

Check HTTP timing metrics with curl

```
curl -o /dev/null -s -w '\
time_namelookup:    %{time_namelookup}s
time_connect:       %{time_connect}s
time_appconnect:    %{time_appconnect}s
time_starttransfer: %{time_starttransfer}s
time_total:         %{time_total}s
' https://example.com
```

6. Sample curl Output (Timed Request)

```
$ curl -o /dev/null -s -w '\
time_namelookup:    %{time_namelookup}s
time_connect:       %{time_connect}s
time_appconnect:    %{time_appconnect}s
time_starttransfer: %{time_starttransfer}s
time_total:         %{time_total}s
' https://api.example.com
```

Example Output:

```
time_namelookup:  0.023s
time_connect:     0.045s
time_appconnect:  0.120s
time_starttransfer: 1.450s
time_total:       1.520s
```

What Each Value Means	
time_namelookup	Time to resolve DNS
time_connect	Time to establish TCP connection
time_appconnect	Time for TLS handshake (HTTPS)
time_starttransfer	Time to receive first byte from server
time_total	Total time for the request

7. Interview Challenge

Your API becomes slow, but CPU and memory look healthy. How do you investigate network-related causes?

8. Troubleshooting Steps (Safe Sequence)

1. Verify DNS resolution time (dig or curl timing).
2. Test basic connectivity (ping or traceroute, but do not rely only on ICMP).
3. Check TCP connection and latency (ss -ti, curl time_connect).
4. Look for packet loss and retransmissions (ss -s, tcpdump, mtr).
5. Test TLS handshake time (curl time_appconnect).
6. Measure application response time (curl time_starttransfer, time_total).
7. Compare with another network or region (client from different location).
8. Review network device logs, firewall, and load balancer metrics.

9. Common Issues and Causes

Issue	Possible Cause	What to Check
High latency (RTT)	Network congestion, long distance, routing issue	ping, traceroute, mtr, load balancer metrics
Packet loss	Unstable link, device issue, congestion	mtr, tcpdump, provider status
TCP retransmissions	Packet loss, overloaded server, firewall	ss -ti, tcpdump, network device logs
Connection timeout	Firewall blocking, network drop, slow server	curl -v, security groups, firewall rules
Slow TLS handshake	Certificate issue, high latency	curl timing, openssl s_client, network latency
Slow application response	Backend dependency, inefficient queries	curl time_starttransfer, application logs, APM

10. Key Takeaways

- Use a structured diagnostic sequence instead of random changes.
- Understand what each tool proves and what it cannot prove.
- Do not assume ICMP (ping) performance represents application performance.
- Look at latency, packet loss, jitter, and TCP retransmissions together.
- Use curl timing, ss -ti, and packet evidence to identify where the delay occurs.
- Compare results from different networks or regions.
- Investigate both network and application dependencies.

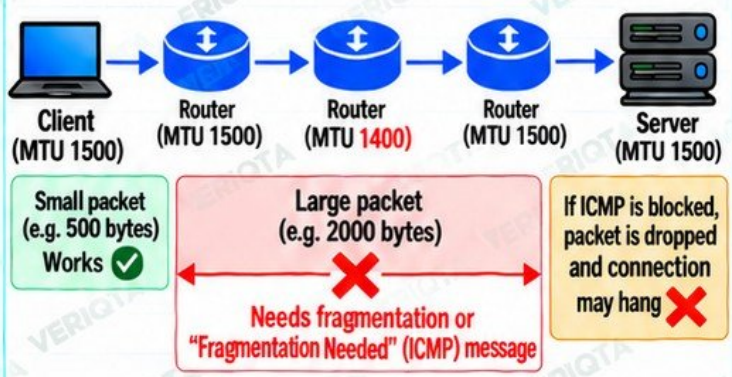
18 MTU, Fragmentation, and Mysterious Connection Failures

VERIQTA

1. Key Concepts

- **MTU (Maximum Transmission Unit):** largest packet size that can be sent on a network link without fragmentation.
- **Path MTU Discovery (PMTUD):** finds the smallest MTU along the path to the destination.
- **TCP MSS (Maximum Segment Size):** TCP segment size, usually MTU – 40 bytes (IP + TCP headers).
- **Fragmentation:** splitting packets when they are larger than the MTU.
- **ICMP filtering:** blocking "Fragmentation Needed" messages can break PMTUD.
- **Why issues happen:** some requests work (small packets) but larger requests fail or hang due to MTU/fragmentation problems.

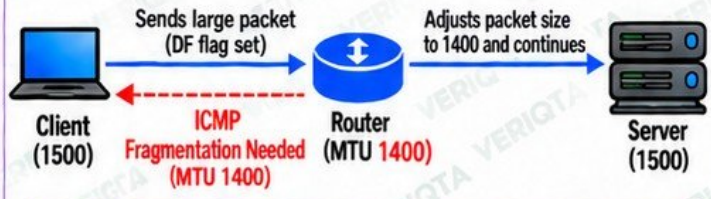
2. How MTU and Fragmentation Work (Flow)



3. MTU, MSS and Headers

Item	Typical Size (Bytes)	Description
Ethernet MTU	1500	Common MTU on most networks
IP header (IPv4)	20	IP header size (no options)
TCP header	20	TCP header size (no options)
TCP MSS (default)	1460	MTU – 40 (20 IP + 20 TCP)
Jumbo frame MTU	9000	Used in some data center networks

4. Path MTU Discovery (PMTUD)



If ICMP messages are blocked, the client never learns the smaller MTU, which can cause timeouts, hangs, or very slow transfers.

5. Common Symptoms

- Small requests work (e.g. small API responses).
- Large responses hang or timeout.
- HTTPS/TLS handshake may succeed, but data transfer stalls.
- Slow downloads or file uploads.
- Issues happen only between specific networks or regions.
- Often caused by MTU mismatch or blocked ICMP (Fragmentation Needed).

6. Practical Commands

1	Check interface MTU	`ip link show`
2	Check route MTU	`ip route get 8.8.8.8`
3	Test packet size (no fragment)	`ping -M do -s 1472 8.8.8.8`
4	Trace path and MTU	`tracepath example.com`
5	Check TCP MSS in connection	`ss -ti dst example.com`
6	Capture ICMP and large packets	`sudo tcpdump -ni any icmp or 'ip[6] & 0x02 != 0'`

7. Interview Challenge

Small API responses succeed, but large HTTPS responses stall.
What network problem might explain this?
How would you investigate it?

8. Example: Finding the Correct MTU

```
# Find the largest working packet size
ping -M do -s 1472 example.com
# If it fails, try a smaller size
ping -M do -s 1400 example.com
# Trace the path MTU
tracepath example.com
```

Example Output (tracepath):

```
1:  192.168.1.1     1.234 ms
2:  10.0.0.1        5.678 ms
3:  203.0.113.5     20.112 ms  pmtu 1400
4:  example.com     21.334 ms
```

pmtu 1400 means the path MTU is 1400 bytes.

9. Common Issues and Causes

Issue	Possible Cause	What to Check
Large responses hang	MTU mismatch or blocked ICMP	Test with ping -M do, tracepath, check firewall rules
HTTPS works but downloads fail	Fragmentation issues	Check path MTU, ICMP filtering
Only some networks affected	Different MTU along path (e.g. VPN, cloud, partner)	Trace path, compare MTU values
Intermittent timeouts	Network devices dropping fragments	Check router/firewall settings
Works with small packets	PMTUD failure	Allow ICMP (Fragmentation Needed), adjust MTU/MSS if needed

10. Key Takeaways

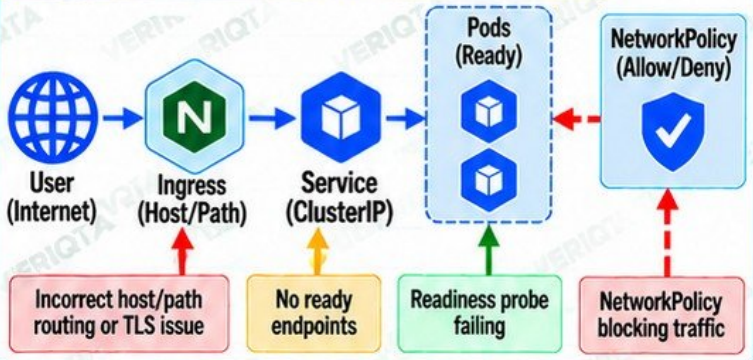
- MTU is the largest packet size that can be sent without fragmentation.
- PMTUD relies on ICMP "Fragmentation Needed" messages.
- If ICMP is blocked, large packets can be dropped and connections may hang.
- Small requests can succeed while large responses fail due to MTU issues.
- Use ping.FCP-MSS, ss, and tcpdump to investigate packet size and MTU.
- Check MTU on interfaces and routes, and understand network path behavior.
- Avoid making unsafe production changes; diagnose and confirm the issue first.
- Adjust MTU/MSS only when necessary and with proper testing.

19 Production Incident: Deployment Succeeded, Traffic Failed

1. Incident Scenario

You deploy a new version. CI/CD reports success, Pods are Running, but users receive 503 responses. You need to find the real cause and restore service.

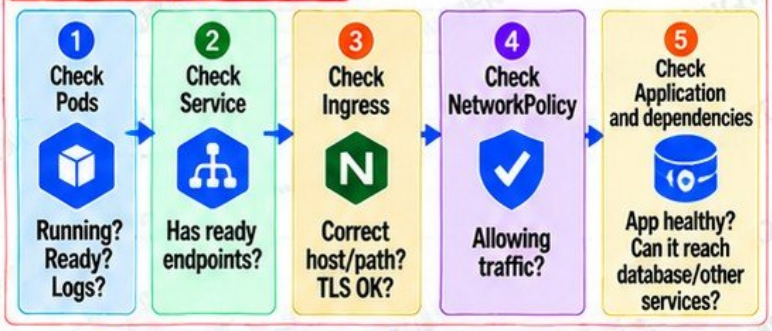
2. Architecture and Possible Failure Points



3. Common Causes of 503 After Deployment

1. Pods are Running but not Ready (readiness probe failing).
2. Service has no ready endpoints.
3. Ingress controller routing to wrong service or path.
4. TLS/certificate or host configuration issue.
5. NetworkPolicy blocking traffic between ingress and pods.
6. Application starts slowly and fails readiness checks.
7. Database or external dependency unreachable.
8. New version has a configuration or environment issue.

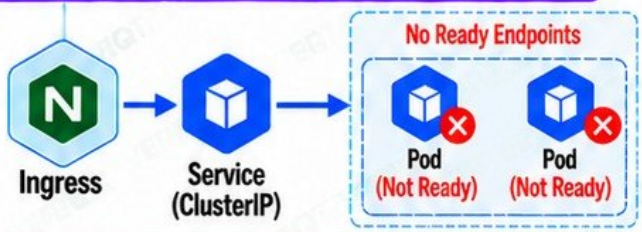
4. Investigation Flow



5. Useful Commands

1	Check pod status	`kubectl get pods -n <namespace>`
2	Describe pod (readiness events)	`kubectl describe pod <pod> -n <ns>`
3	View pod logs	`kubectl logs <pod> -n <ns>`
4	Check service and endpoints	`kubectl get svc, endpointslices -n <ns>`
5	Describe service	`kubectl describe svc <service> -n <ns>`
6	Check ingress	`kubectl get ingress -n <ns>`
7	Describe ingress	`kubectl describe ingress <ingress> -n <ns>`
8	Check network policies	`kubectl get networkpolicy -n <ns>`
9	Test from inside cluster	`kubectl run -it --rm test --image=curlimages/curl -- curl -I http://<service>`

6. Example: Service With No Ready Endpoints



The Service has no ready endpoints, so the load balancer returns 503 (Service Unavailable) even though the Pods are Running.

```
kubectl get endpointslices <service> -n <ns>
NAME          ADDRESSES   PORTS   CONDITIONS
web-svc-abc   <none>      80      ready: false
```

7. Interview Challenge

CI/CD reports success, Pods are Running, but users receive 503 responses. What evidence do you collect, and when do you roll back?

8. Rollback Decision Guide

Evidence	Roll Back?
No ready endpoints after fixes	Yes (if cannot quickly fix)
Ingress misconfiguration	Fix, not rollback
NetworkPolicy blocking traffic	Fix, not rollback
Application bug in new version	Yes (if confirmed and no quick fix)
External dependency issue	Usually no (fix dependency)
Widespread 503 impacting users	Yes (if fix not immediate)

9. Key Takeaways

- A successful deployment does not mean the service is available.
- Always verify pod readiness, service endpoints, and ingress routing.
- Check network policies and external dependencies.
- Use evidence before deciding to roll back.
- Automate health checks and monitoring to detect issues early.
- Rollback when the impact is user-facing and cannot be resolved quickly.

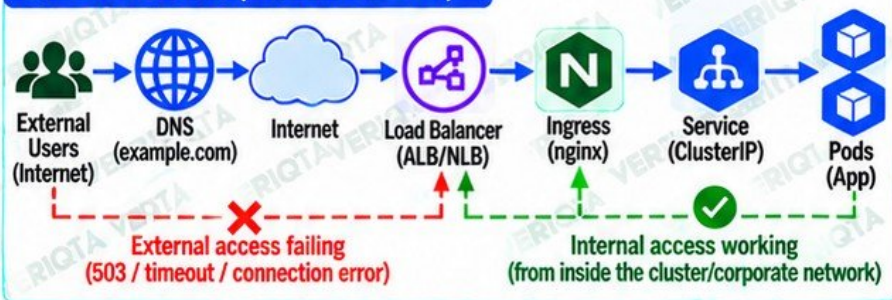
VERIQTA

20 The DevOps Networking Interview: Final Troubleshooting Challenge

1. Scenario Overview

A service works internally but fails for external customers. Users outside the company cannot access the application, while internal users can. You must investigate the full network path, find the root cause, restore service, and prevent recurrence.

2. Architecture (End-to-End Flow)



3. Investigation Flow

1. Reproduce the issue and confirm scope (external vs internal).
2. Check DNS resolution and records.
3. Verify network routing and reachability.
4. Test TCP connectivity (ports, firewalls, security groups).
5. Validate TLS/SSL configuration.
6. Check load balancer health and target status.
7. Inspect ingress controller and routing rules.
8. Verify Kubernetes service, endpoints, and pod health.
9. Check application logs and dependencies.
10. Identify the root cause and implement a safe fix.
11. Verify recovery from external and internal locations.

4. Common Commands by Layer

DNS	`dig example.com`	`nslookup example.com`
Network Connectivity	`ping example.com`	`traceroute example.com`
Port / TCP	`nc -zv example.com 443`	`ss -tulnp | grep :443`
TLS / SSL	`openssl s_client -connect example.com:443`	`curl -vI https://example.com`
Load Balancer	`aws elbv2 describe-target-health --target-group-arn <arn>`	
Kubernetes	`kubectl get ingress,svc,endpointslices,pods -n <ns>`	`kubectl describe ingress <name>`
Application	`kubectl logs <pod> -n <ns>`	`curl -v http://<service>.<ns>.svc.cluster.local`

5. Example Investigation (Step by Step)

Step		Command / Check	What It Proves
1	DNS resolution	dig example.com	Returns correct public IP
2	Network path	traceroute example.com	Finds where traffic drops
3	TCP connectivity	nc -zv example.com 443	Confirms port is open
4	TLS handshake	openssl s_client -connect example.com:443	Certificate and handshake work
5	Load balancer	Check target health	Targets are healthy
6	Ingress routing	kubectl describe ingress	Host/path rules are correct
7	Service endpoints	kubectl get endpointslices	Pods are registered
8	Pod health	kubectl get pods	Pods are ready
9	Application logs	kubectl logs <pod>	Finds app errors or dependency issues
10	Verify externally	curl -I https://example.com	Confirms service is working

6. Example Root Causes

- DNS pointing to wrong IP
- Security group or firewall blocking traffic
- Load balancer targets unhealthy
- Ingress rule misconfiguration (host/path)
- TLS certificate issue or expired cert
- Service with no ready endpoints
- NetworkPolicy blocking traffic
- Application failing health checks
- External dependency unreachable
- MTU/fragmentation or routing issue

7. Safe Mitigation Steps

1. Fix DNS record or routing
2. Update security group rules (minimal and correct)
3. Restore healthy targets
4. Correct ingress rules
5. Renew/replace TLS certificate
6. Fix readiness probe or application issue
7. Adjust NetworkPolicy (least privilege)
8. Roll back to previous version (if quick resolution needed)
9. Verify recovery from external location
10. Monitor to ensure stability

8. Interview Challenge

A service works internally but fails for external customers. Explain your investigation from first symptom to verified recovery. What evidence do you collect, and when do you roll back?

9. Assessment Criteria

- Form clear hypotheses.
- Select the right commands.
- Interpret evidence and explain what it means.
- Identify the most likely root cause.
- Choose safe and minimal mitigation.
- Verify recovery from external and internal locations.
- Explain how to prevent the issue in the future.

10. Key Takeaways

- Always test from both internal and external locations.
- Check each layer: DNS → routing → TCP → TLS → load balancer → ingress → service → pods → application.
- A successful deployment does not guarantee service availability.
- Use evidence, not assumptions, and make minimal, safe changes.
- Implement monitoring and alerts to detect similar issues early.